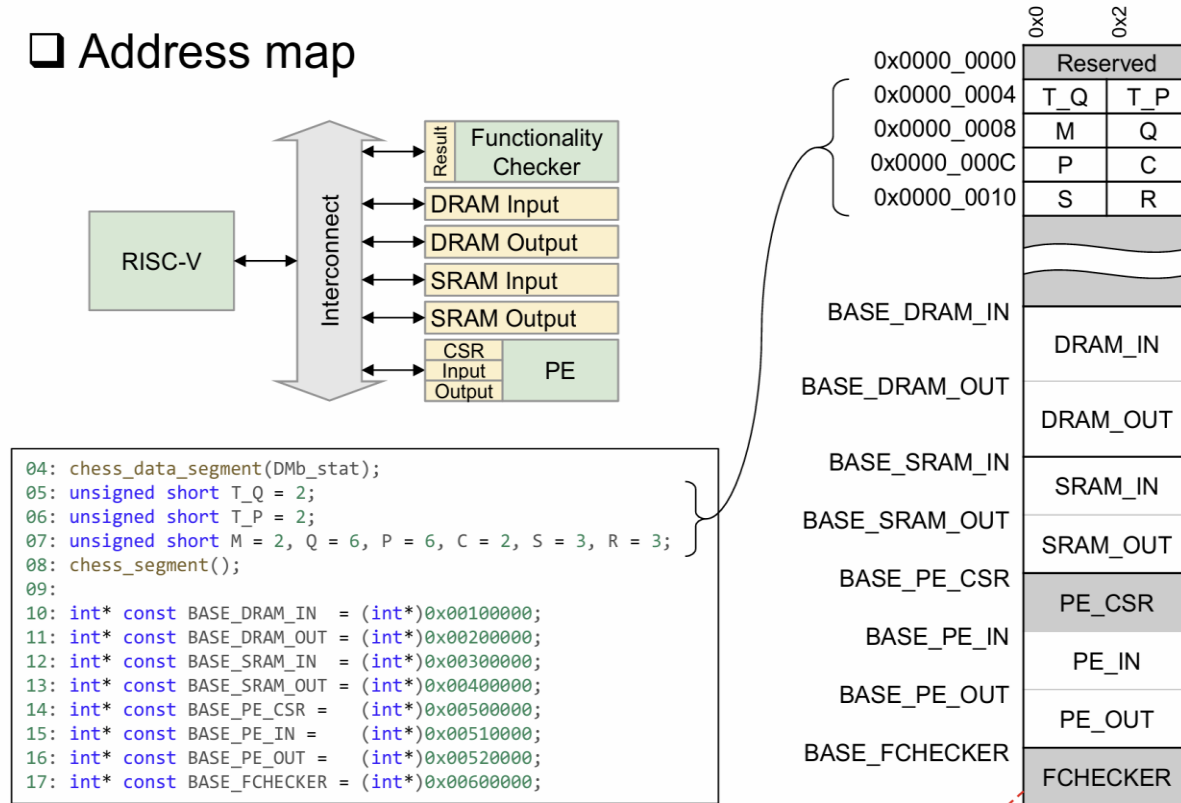


outline

- Introduction
 - 간단한 개념 리뷰
- Baseline
 - c code review
- Assembly_code review
 - idea1 to 12
- Performance of Optimization Ideas

introduction

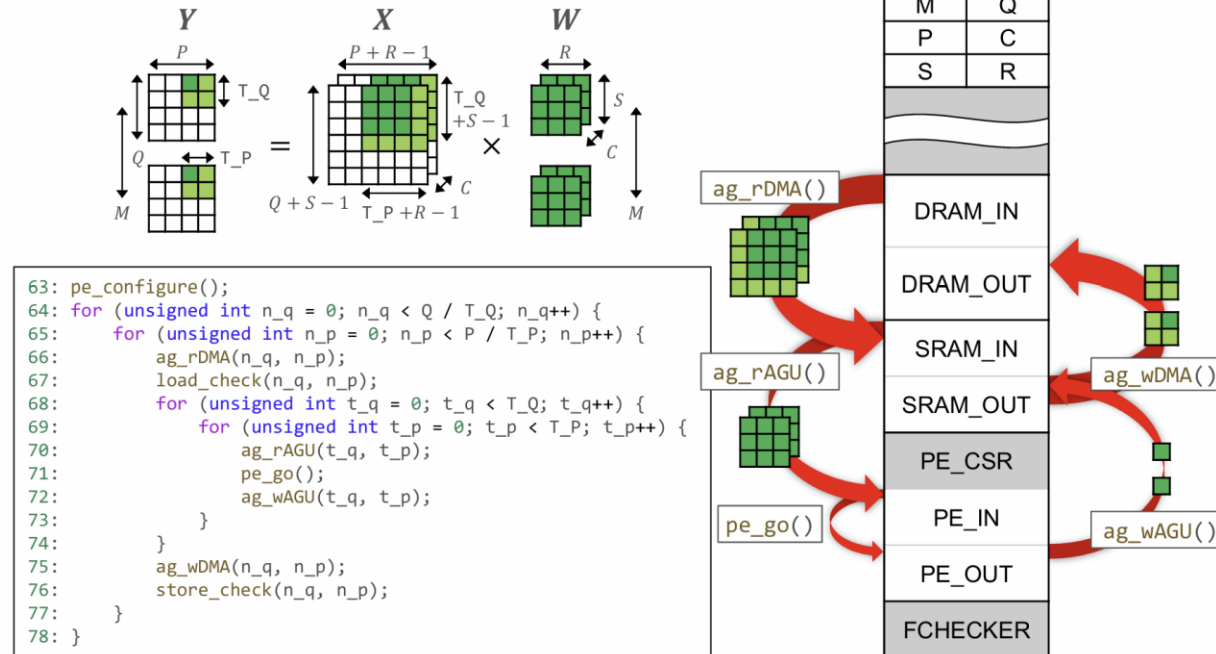
□ Address map



- RISC-V는 memory-mapped addres를 통해 DRAM, SRAM, PE를 제어한다.
- 각 하드웨어 블록은 고정된 주소 영역에 매핑됨
- 데이터 흐름: DRAM → SRAM → PE → SRAM → DRAM
- 최적화 대상: ag_* 함수의 주소 생성 코드

introduction

□ Main program



- Main program은 tile 단위로 데이터를 이동시키고 PE 연산을 반복 수행한다.
- ag_rDMA : DRAM $\rightarrow$ SRAM
- ag_rAGU : SRAM $\rightarrow$ PE input
- pe_go : PE 연산 수행
- ag_wAGU , ag_wDMA : 결과 저장

introduction

□ Loop tiling/ordering

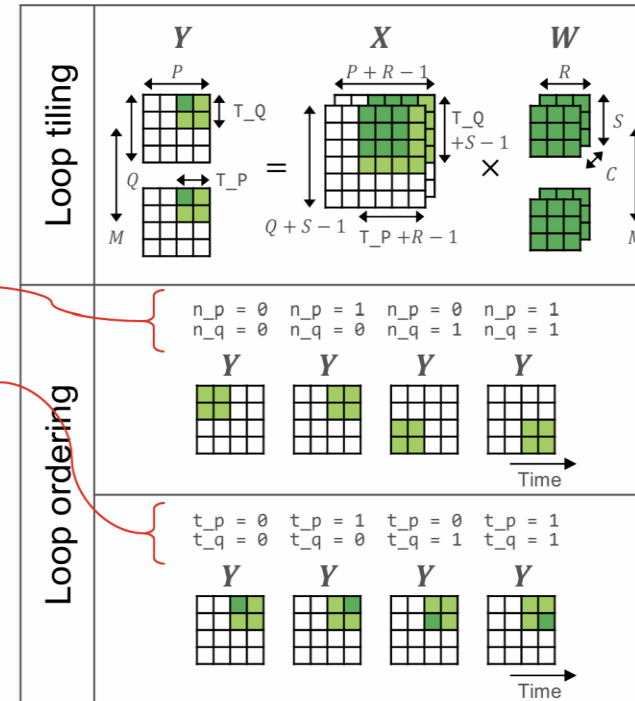
- (p, q) tiling specified below

E.g.,

$P = 4, Q = 4, R = 3, S = 3, C = 2, M = 2$

$T_P = 2, T_Q = 2$

```
63: pe_configure();
64: for (unsigned int n_q = 0; n_q < Q / T_Q; n_q++) {
65:     for (unsigned int n_p = 0; n_p < P / T_P; n_p++) {
66:         ag_rDMA(n_q, n_p);
67:         load_check(n_q, n_p);
68:         for (unsigned int t_q = 0; t_q < T_Q; t_q++) {
69:             for (unsigned int t_p = 0; t_p < T_P; t_p++) {
70:                 ag_rAGU(t_q, t_p);
71:                 pe_go();
72:                 ag_wAGU(t_q, t_p);
73:             }
74:         }
75:         ag_wDMA(n_q, n_p);
76:         store_check(n_q, n_p);
77:     }
78: }
79: printf("Functionality %s\n", fcheck() ? "failed" : "passed");
```



- Loop tiling은 출력 feature map을 작은 tile로 나누어 처리하는 방식이다.
 - n_q, n_p : tile 위치 결정
 - t_q, t_p : tile 내부 pixel 순회
 - 반복 호출되는 ag_* 함수 최적화가 성능 향상 핵심

baseline

```
87 void ag_rDMA(unsigned int n_q, unsigned int n_p)
88 {
89     unsigned int input_w = P + R - 1;
90     unsigned int tile_h = T_Q + S - 1;
91     unsigned int tile_w = T_P + R - 1;
92
93     unsigned int start_q = n_q * T_Q;
94     unsigned int start_p = n_p * T_P;
95
96     unsigned int row_len = tile_w * C; // 한 tile row에
97     unsigned int dram_stride = input_w * C; // DRAM에서 한
98
99     volatile int *src_base =
100         BASE_DRAM_IN + ((start_q * input_w + start_p) * C);
101
102     volatile int *dst_base = BASE_SRAM_IN;
103
104     for (unsigned int q = 0; q < tile_h; q++) {
105         volatile int *src = src_base + q * dram_stride;
106         volatile int *dst = dst_base + q * row_len;
107
108         unsigned int k = 0;
109
110         // 4개씩 복사
111         for (; k + 3 < row_len; k += 4) {
112             dst[k] = src[k];
113             dst[k + 1] = src[k + 1];
114             dst[k + 2] = src[k + 2];
115             dst[k + 3] = src[k + 3];
116         }
117
118         // 남은 것 처리
119         for (; k < row_len; k++) {
120             dst[k] = src[k];
121         }
122     }
123 }

128 void ag_rAGU(unsigned int t_q, unsigned int t_p)
129 {
130     unsigned int tile_w = T_P + R - 1;
131
132     unsigned int row_len = R * C; // kernel 한 row에서
133     unsigned int src_stride = tile_w * C;
134
135     volatile int *dst_base = BASE_PE_IN;
136
137     for (unsigned int s = 0; s < S; s++) {
138         volatile int *src =
139             BASE_SRAM_IN + (((t_q + s) * tile_w + t_p) * C);
140
141         volatile int *dst =
142             dst_base + s * row_len;
143
144         unsigned int k = 0;
145
146         // 4개씩 복사
147         for (; k + 3 < row_len; k += 4) {
148             dst[k] = src[k];
149             dst[k + 1] = src[k + 1];
150             dst[k + 2] = src[k + 2];
151             dst[k + 3] = src[k + 3];
152         }
153
154         // 남은 것 처리
155         for (; k < row_len; k++) {
156             dst[k] = src[k];
157         }
158     }
159 }

164 void ag_wAGU(unsigned int t_q, unsigned int t_p)
165 {
166     volatile int *src = BASE_PE_OUT;
167     volatile int *dst = BASE_SRAM_OUT + ((t_q * T_P + t_p) * M);
168
169     unsigned int k = 0;
170
171     // 4개씩 복사
172     for (; k + 3 < M; k += 4) {
173         dst[k] = src[k];
174         dst[k + 1] = src[k + 1];
175         dst[k + 2] = src[k + 2];
176         dst[k + 3] = src[k + 3];
177     }
178
179     // 남은 것 처리
180     for (; k < M; k++) {
181         dst[k] = src[k];
182     }
183 }

void ag_wDMA(unsigned int n_q, unsigned int n_p)
{
    unsigned int start_q = n_q * T_Q;
    unsigned int start_p = n_p * T_P;
    unsigned int row_len = T_P * M; // tile output 한
    unsigned int dram_stride = P * M;
    volatile int *src_base = BASE_SRAM_OUT;
    volatile int *dst_base =
        BASE_DRAM_OUT + ((start_q * P + start_p) * M);
    for (unsigned int q = 0; q < T_Q; q++) {
        volatile int *src = src_base + q * row_len;
        volatile int *dst = dst_base + q * dram_stride;
        unsigned int k = 0;
        // 4개씩 복사
        for (; k + 3 < row_len; k += 4) {
            dst[k] = src[k];
            dst[k + 1] = src[k + 1];
            dst[k + 2] = src[k + 2];
            dst[k + 3] = src[k + 3];
        }
        // 남은 것 처리
        for (; k < row_len; k++) {
            dst[k] = src[k];
        }
    }
}
```

C코드 결과값

```
[sweep] T = (1/N) sum E_n/ALB_n = 6.1545 (N=168)
[sweep] finished: 168/168 passed, 168/168 completed
Warning: some scml2 stream(s) not destroyed before infrastructure shutdown. Set
environment variable SNPS_SLS_VP_SCML2_LOGGING_VERBOSE_WARNINGS for more details
[student14@students mp26_termproject]$ cd simulator/
[student14@students simulator]$ awk -F, 'NR>1 {sum[$1]+=$16; cnt[$1]++;} END {for
(l in sum) printf "Layer %s avg = %.6f\n", l, sum[l]/cnt[l]}' result.csv | sort
-n -k2
Layer 0 avg = 6.154520
Layer 1 avg = 6.116139
Layer 2 avg = 7.486516
Layer 3 avg = 5.084264
Layer 4 avg = 5.831604
Layer 5 avg = 4.560619
Layer 6 avg = 4.974808
Layer 7 avg = 4.270510
Layer 8 avg = 4.442920
[student14@students simulator]$
```

- 각 AGU/DMA 함수는 tile 좌표(n_q, n_p 또는 t_q, t_p)를 기준으로 src/dst 시작 주소를 계산
- rDMA는 DRAM input을 SRAM input으로, rAGU는 SRAM input을 PE input으로 복사
- wAGU는 PE output을 SRAM output으로, wDMA는 SRAM output을 DRAM output으로 저장
- row_len과 stride를 이용해 row 단위로 이동하면서 필요한 데이터만 연속적으로 복사

Assembly_code review

Idea1: 4개 AGU/DMA 루틴을 assembly로 재작성

ag_rDMA.s

```
1  .text
2  ▾ global 0 ag_rDMA
3
4      lhu    x4, 4(x0)
5      lhu    x6, 18(x0)
6      mul    x5, x4, x10
29     lui    x5, 256
30     sh2add  x11, x11, x5
43     ▾ rdma_copy_loop:
44         lw      x16, 0(x15)
45         lw      x17, 4(x15)
46         lw      x18, 8(x15)
47         lw      x19, 12(x15)
48
49         sw      x16, 0(x14)
50         sw      x17, 4(x14)
51         sw      x18, 8(x14)
52         sw      x19, 12(x14)
53
54         addi    x15, x15, 16
55         addi    x14, x14, 16
56         addi    x13, x13, 4
57         bltu    x13, x10, rdma_copy_loop
```

Result screenshot

```
Functionality passed
[168/168] 1.8 T_0=7 T_P=7 M=64 O=7 P=7 C=32 S=1 R=1 |
[sweep] T = (1/N) sum E n/ALB n = 5.2102 (N=168)
[sweep] finished: 168/168 passed, 168/168 completed
```

T=5.2102

-기존 baseline과 generic 연산 구조를 rAGU/rDMA/wAGU/ wDMA assembly 루틴으로 직접 재작성

-주소 계산과 load/store 흐름을 명령어 단위로 제어하여 불필요한 control overhead 감소

Idea	Speedup (culmulative)	T
Baseline	1x	6.1545
---	---	---
---	---	---
Idea1 (this slide)	1.18x (1.18)	5.2102

Assembly_code review

Idea2: rDMA/wDMA copy loop를 4-word 단위로 unroll

ag_rDMA.s

```
43  ◁ rdma_copy_loop:
44      lw      x16, 0(x15)
45      lw      x17, 4(x15)
46      lw      x18, 8(x15)
47      lw      x19, 12(x15)
48
49      sw      x16, 0(x14)
50      sw      x17, 4(x14)
51      sw      x18, 8(x14)
52      sw      x19, 12(x14)
53
54      addi    x15, x15, 16
55      addi    x14, x14, 16
56      addi    x13, x13, 4
57      bltu    x13, x10, rdma_copy_loop
```

-memory copy를 1개 word씩 처리하지 않고 4개 word 단위로 묶어서 load/store 수행

-반복 횟수, 주소 증가 명령어, loop branch 횟수를 줄여 DMA copy 비용 감소

Result screenshot

```
Functionality passed
[168/168] L8 T Q=7 T P=7 M=64 Q=7 P=7 C=32 S=1 R=1 |
[sweep] T = (1/N) sum E n/ALB n = 4.7937 (N=168)
[sweep] finished: 168/168 passed, 168/168 completed
```

T=4.7937

Idea	Speedup (culmulative)	T
Baseline	1x	6.1545
---	---	---
Idea1	1.18x (1.18)	5.2102
Idea2 (this slide)	1.08x (1.28)	4.7937

Assembly_code review

Idea3: M/C/P 값별 unrolled fast path 추가

ag_wAGU.s

```
14      addi x7, x3, -4
15      beqz x7, wagu_m4_fast
16
17      addi x7, x3, -8
18      beqz x7, wagu_m8_fast
19
20      addi x7, x3, -16
21      beqz x7, wagu_m16_fast
22
23      addi x7, x3, -32
24      beqz x7, wagu_m32_fast
```

-M=4/8/16/32/64 처럼 자주 등장하는 출력 채널 수를 전용 경로로 분기
-각 fast path에서 load/store를 하드코딩하여 inner loop와 반복 branch를 제거

Result screenshot

```
Functionality passed
[168/168] L8 T_0=7 T_P=7 M=64 O=7 P=7 C=32 S=1 R=1 |
[sweep] T = (1/N) sum E_n/ALB_n = 4.1600 (N=168)
[sweep] finished: 168/168 passed, 168/168 completed
```

T=4.16

Idea	Speedup (culmulative)	T
Baseline	1x	6.1545
Idea1	1.18x (1.18)	5.2102
Idea2	1.08x (1.28)	4.7937
Idea3 (this slide)	1.15x (1.479)	4.16

Assembly_code review

Idea4: rDMA의 W=8/16/32 전용 fast path 추가

ag_rDMA.s

```
38      addi    x7, x10, -8
39      beqz    x7, rdma_w8_fast
40
41      addi    x7, x10, -16
42      beqz    x7, rdma_w16_fast
43
44      addi    x7, x10, -32
45      beqz    x7, rdma_w32_fast
46
47      j       rdma_normal
```

```
137      v rdma_w16_fast:
138      |      li      x12, 0
139
140      v rdma_w16_row_loop:
141      |      mv      x14, x6
142      |      mv      x15, x11
143
144      |      lw      x16, 0(x15)
145      |      lw      x17, 4(x15)
146      |      lw      x18, 8(x15)
147      |      lw      x19, 12(x15)
148      |      lw      x20, 16(x15)
149      |      lw      x21, 20(x15)
150      |      lw      x22, 24(x15)
151      |      lw      x23, 28(x15)
152      |      lw      x24, 32(x15)
153      |      lw      x25, 36(x15)
154      |      lw      x26, 40(x15)
155      |      lw      x27, 44(x15)
156      |      lw      x28, 48(x15)
157      |      lw      x29, 52(x15)
158      |      lw      x30, 56(x15)
159      |      lw      x31, 60(x15)
```

Result screenshot

```
=====
Evaluation finished
Total tests = 168
Passed tests = 168
Failed tests = 0
Result CSV = /home/student14/Workspace-isee/term_p
Final T average = 4.088147
[student14@students mp26_termproject]$ awk -F, 'NR>
1, sum[l]/cnt[l]]' eval_result.csv | sort -n -k2
Layer 1 avg = 3.751638
Layer 2 avg = 5.169757
Layer 3 avg = 3.596496
Layer 4 avg = 3.885087
Layer 5 avg = 3.184385
Layer 6 avg = 3.151573
Layer 7 avg = 2.914667
Layer 8 avg = 2.682610
```

T=4.0881

-feature map width가 8/16/32인 경우를 generic loop로 처리
하지 않고 전용 copy 경로로 분리
-row 단위 copy를 하드코딩하여 layer2에서 크게 발생하던
rDMA 병목을 완화

Idea	Speedup (culmulative)	T
Baseline	1x	6.1545
Idea2	1.08x (1.28)	4.7937
Idea3	1.15x (1.479)	4.16
Idea4 (this slide)	1.01x (1.505)	4.0881

Assembly_code review

Idea5: mul 연산을 shift 기반 주소 계산으로 치환

ag_wDMA.s

```
18      addi    x7, x5, -8
19      beqz    x7, wdma_shift8
20
21      addi    x7, x5, -4
22      beqz    x7, wdma_shift4
23
24      addi    x7, x5, -16
25      beqz    x7, wdma_shift16
26
27      addi    x7, x5, -32
28      beqz    x7, wdma_shift32
29
30      j        wdma_shift64
```

-M/P/C 값이 4/8/16/32/64처럼 2의 거듭제곱인 경우 mul 대신 slli 사용
-곱셈 latency를 줄이고 sh2add를 활용해 base+offset 주소 계산을 단순화

Result screenshot

```
=====
Evaluation finished
Total tests = 168
Passed tests = 168
Failed tests = 0
Result CSV = /home/student14/Workspace-jslee/term
result.csv
Final T average = 4.007178
[student14@students mp26_termproject]$ awk -F, 'NR
2; cnt[$1]++} END {for (l in sum) printf "Layer %s
l]}' eval_result.csv | sort -n -k2
Layer 1 avg = 3.693437
Layer 2 avg = 5.007160
Layer 3 avg = 3.561366
Layer 4 avg = 3.829105
Layer 5 avg = 3.177278
Layer 6 avg = 3.108760
Layer 7 avg = 2.904838
Layer 8 avg = 2.635841
[student14@students mp26_termproject]$
```

T=4.0072

Idea	Speedup (culmulative)	T
Baseline	1x	6.1545
Idea3	1.15x (1.479)	4.16
Idea4	1.01x (1.505)	4.0881
Idea5 (this slide)	1.02x (1.535)	4.0072

Assembly_code review

Idea6: 3×3 C=8/16/32 전용 unrolled fast path를 추가

기존:

ag_rAGU.s

```
addi x7, x6, -4
beqz x7, ragu_3x3_c4_fast
```

변경:

```
1  addi x7, x6, -4
2  bnez x7, ragu_check_c8
3  j    ragu_3x3_c4_fast
4
5  ragu_check_c8:
6  ...
7  j    ragu_3x3_c8_fast
8
9  ragu_check_c16:
10 ...
11 j    ragu_3x3_c16_fast
12
13 ragu_check_c32:
14 ...
15 j    ragu_3x3_c32_fast
16
```

- C=8/16/32도 전용 경로로 분기하여 주소 계산과 loop branch를 제거하고, load/store를 하드코딩 수준으로 전개

Result screenshot

```
Evaluation finished
Total tests = 168
Passed tests = 168
Failed tests = 0
Result CSV = /home/student14/Workspace-jslee/term_project
result.csv
Final T average = 3.872569
[student14@students mp26_termproject]$ awk -F, 'NR>1 && $
2; cnt[$1]++} END {for (l in sum) printf "Layer %s avg =
l]}' eval_result.csv | sort -n -k2
Layer 1 avg = 3.693437
Layer 2 avg = 5.007160
Layer 3 avg = 2.884498
Layer 4 avg = 3.829105
Layer 5 avg = 2.606229
Layer 6 avg = 3.108760
Layer 7 avg = 2.351612
Layer 8 avg = 2.635841
[student14@students mp26_termproject]$
```

T=3.8725

Idea	Speedup (culmulative)	T
Baseline	1x	6.1545
Idea4	1.01x (1.50)	4.0881
Idea5	1.02x (1.53)	4.0072
Idea6 (this slide)	1.03x (1.58)	3.8725

Assembly_code review

Idea7: rDMA/wDMA에 대형 row_len exact fast path를 추가

ag_rDMA.s

```
ag_rDMA

addi x7, x10, -32
bnez x7, rdma_check_w56
j rdma_w32_fast

rdma_check_w56:
// dispatch to W=56 exact path
j rdma_w56_fast

rdma_check_w64:
// dispatch to W=64 exact path
j rdma_w64_fast

rdma_check_w112:
// dispatch to W=112 exact path
j rdma_w112_fast

rdma_check_w224:
// dispatch to W=224 exact path
j rdma_w224_fast
```

ag_wDMA.s

```
ag_wDMA

wdma_prepare4:
...
addi x7, x4, -56
bnez x7, wdma_p4_check112
j wdma_rowlen56_fast

wdma_p4_check112:
// dispatch to row_len=112 exact path
j wdma_rowlen112_fast

wdma_p4_check224:
// dispatch to row_len=224 exact path
j wdma_rowlen224_fast
```

- rDMA는 기존 W=8/16/32 중심에서 W=56/64/112/224 같은 큰 width 전용 경로를 추가

- wDMA는 row_len=56/112/224/448 exact path와 P=16 계열 전용 경로를 추가해서 loop branch와 반복 copy 비용절감

Result screenshot

```
Evaluation finished
Total tests = 168
Passed tests = 168
Failed tests = 0
Result CSV = /home/student14/Workspace-jslee/term_project/eval_result.csv
Final T average = 3.810536
[student14@students mp26_termproject]$ awk -F, 'NR>1 {sum[l]+=$1; cnt[l]++;} END {for (l in cnt) printf "%s\t%.6f\n", l, sum[l]/cnt[l]}' eval_result.csv | sort -n -k2
Layer 1 avg = 3.660551
Layer 2 avg = 4.938182
Layer 3 avg = 2.884938
Layer 4 avg = 3.690712
Layer 5 avg = 2.537226
Layer 6 avg = 2.975092
Layer 7 avg = 2.346270
Layer 8 avg = 2.483843
[student14@students mp26_termproject]$
```

T=3.8105

Idea	Speedup (culmulative)	T
Baseline	1x	6.1545
Idea5	1.02x (1.53)	4.0072
Idea6	1.03x (1.58)	3.8725
Idea7 (this slide)	1.01x (1.61)	3.8105

Assembly_code review

Idea8: wDMA contiguous copy path & rDMA Layer2 W=8, H=2 exact path

ag_rDMA.s

```
ag_rDMA
rdma_w8_fast:
// dispatch to row_len=8, tile_h=2 exact path
addi x7, x4, -2
beqz x7, rdma_w8_h2_fast
...
rdma_w8_h2_fast:
// exact path for W=8, H=2
// fully unrolled 2-row copy
ret
```

ag_wDMA.s

```
ag_wDMA
wdma_shift4:
beq x4, x10, wdma_contiguous_prepare
wdma_shift8:
beq x4, x10, wdma_contiguous_prepare
wdma_shift16:
beq x4, x10, wdma_contiguous_prepare
wdma_shift32:
beq x4, x10, wdma_contiguous_prepare
wdma_shift64:
beq x4, x10, wdma_contiguous_prepare
...
wdma_contiguous_prepare:
// dispatch to contiguous copy path
// T_P == P
// SRAM_OUT and DRAM_OUT rows are contiguous
j wdma_contiguous_check32
...
wdma_contiguous_check32:
// copy 32 words at once
wdma_contiguous_check16:
// copy 16 words at once
wdma_contiguous_check8:
// copy 8 words at once
wdma_contiguous_check4:
// copy remaining 4 words
```

-wDMA에서 SRAM/DRAM row 길이가 같은 경우
바로 복사하는 contiguous 경로 추가.

- Layer2에서 자주 등장하는 row_len=8, tile_h=2 케이스를
rDMA 전용 exact path로 분리하여 row loop 자체를 제거

Result screenshot

```
Evaluation finished
Total tests = 168
Passed tests = 168
Failed tests = 0
Result CSV = /home/student14/Workspace-jslee/term_projec
Final T average = 3.799222
[student14@students mp26_termproject]$ awk -F, 'NR>1 &&
l, sum[l]/cnt[l]}' eval_result.csv | sort -n -k2
Layer 1 avg = 3.658753
Layer 2 avg = 4.919646
Layer 3 avg = 2.882078
Layer 4 avg = 3.667111
Layer 5 avg = 2.532319
Layer 6 avg = 2.953845
Layer 7 avg = 2.343951
Layer 8 avg = 2.478741
[student14@students mp26_termproject]$ ^[[2~
```

T=3.7992

Idea	Speedup (culmulative)	T
Baseline	1x	6.1545
Idea6	1.03x (1.58)	3.8725
Idea7	1.01x (1.61)	3.8105
Idea8 (this slide)	1.003x (1.62)	3.7992

Assembly_code review

Idea9: 더 많은 width case 추가._generic loop 사용 구간감소시키기

ag_rDMA.s

```
ag_rDMA

rdma_check_w224:
// dispatch to W=224 exact path
j    rdma_w224_fast

rdma_check_w28:
// dispatch to W=28 exact path
j    rdma_w28_fast

rdma_check_w24:
// dispatch to W=24 exact path
j    rdma_w24_fast

rdma_check_w36:
// dispatch to W=36 exact path
j    rdma_w36_fast

rdma_check_w40:
// dispatch to W=40 exact path
j    rdma_w40_fast

rdma_check_w48:
// dispatch to W=48 exact path
j    rdma_w48_fast

rdma_check_w72:
// dispatch to W=72 exact path
j    rdma_w72_fast
```

ag_wDMA.s

```
ag_wDMA

wdma_prepare4:

addi x7, x4, -8
// dispatch to row_len=8 exact path
beqz x7, wdma_p4_w8

addi x7, x4, -16
// dispatch to row_len=16 exact path
beqz x7, wdma_p4_w16

addi x7, x4, -28
// dispatch to row_len=28 exact path
beqz x7, wdma_p4_w28

addi x7, x4, -32
// dispatch to row_len=32 exact path
beqz x7, wdma_p4_w32
```

Result screenshot

```
Evaluation finished
Total tests = 168
Passed tests = 168
Failed tests = 0
Result CSV = /home/student14/Workspace-jslee/term_
Final T average = 3.747962
[student14@students mp26_termproject]$ awk -F, 'NR
l, sum[l]/cnt[l]}' eval_result.csv | sort -n -k2
Layer 1 avg = 3.544810
Layer 2 avg = 4.882157
Layer 3 avg = 2.833671
Layer 4 avg = 3.667111
Layer 5 avg = 2.534255
Layer 6 avg = 2.953845
Layer 7 avg = 2.345161
Layer 8 avg = 2.478741
[student14@students mp26_termproject]$
```

T=3.748

- rDMA: W=24/28/36/40/48/72 fast path 추가.
- wDMA: P=4에서 W=8/16/28/32 전용 경로 추가.

Idea	Speedup (culmulative)	T
Baseline	1x	6.1545
Idea7	1.01x (1.61)	3.8105
Idea8	1.003x (1.62)	3.7992
Idea9 (this slide)	1.013x (1.64)	3.748

Assembly_code review

Idea10: rDMA의 row_len=8 에 tile_h exact path + end-pointer loop를 추가 & rAGU의 S=1 hot path를 앞쪽으로 재배치

ag_rAGU.s

```
ag_rAGU

lhu x5, 16(x0)
addi x7, x5, -1
bnez x7, ragu_not_1x1

// dispatch to S=1 hot path
// inline S=1, C=4 exact copy path
ret

ragu_1x1_check_c8:
// dispatch to S=1, C=8 exact path
j   ragu_c8_fast

ragu_1x1_check_c16:
// dispatch to S=1, C=16 exact path
j   ragu_c16_fast

ragu_1x1_go_c32:
// dispatch to S=1, C=32 exact path
j   ragu_c32_fast
```

ag_rDMA.s

```
ag_rDMA

addi x7, x10, -8
bnez x7, rdma_check_w16

rdma_w8_fast:
// dispatch to W=8 hot path
// tile_h=2 falls through to exact path

rdma_w8_check_h4:
// dispatch to W=8, H=4 exact path
j   rdma_w8_h4_fast

rdma_w8_check_h7:
// dispatch to W=8, H=7 exact path
j   rdma_w8_h7_fast

rdma_w8_check_h8:
// dispatch to W=8, H=8 exact path
j   rdma_w8_h8_fast

rdma_w8_general_prepare:
// dispatch to W=8 general end-pointer loop
j   rdma_w8_ep_loop
```

- rAGU: ag_rAGU는 S==1일 때 바로 1x1 path로 fall-through 되게 변경
- rDMA: W뿐 아니라 H=2/4/7/8/14/28/56같은 height까지 고려한 exact path 추가.

Result screenshot

```
Evaluation finished
Total tests = 168
Passed tests = 168
Failed tests = 0
Result CSV = /home/student14/Workspace-jslee/term_project
result.csv
Final T average = 3.706386
[student14@students mp26_termproject]$ awk -F, 'NR>1 && $
l, sum[l]/cnt[l]}' eval_result.csv | sort -n -k2
Layer 1 avg = 3.586829
Layer 2 avg = 4.687467
Layer 3 avg = 2.852567
Layer 4 avg = 3.657706
Layer 5 avg = 2.543773
Layer 6 avg = 2.970214
Layer 7 avg = 2.350100
Layer 8 avg = 2.499574
[student14@students mp26_termproject]$
```

T=3.7063

Idea	Speedup (culmulative)	T
Baseline	1x	6.1545
Idea8	1.003x (1.62)	3.7992
Idea9	1.013x (1.64)	3.748
Idea10 (this slide)	1.011x (1.66)	3.7063

Assembly_code review

Idea11: fall-through로 재배치 &rDMA width dispatch 순서를 조정

ag_wAGU.s

```
ag_wAGU

addi x7, x3, -8
bnez x7, wagu_not_m8

wagu_m8_fast:
// dispatch to M=8 hot path
// branch not-taken path
ret

wagu_not_m8:
// dispatch to M=4/16/32/64 paths
```

ag_wDMA.s

```
ag_wdMA
    addi x7, x5, -8
    bnez x7, wdma_not_m8

wdma_shift8:
    // dispatch to M=8 hot path
    // branch not-taken path
    j     wdma_prepare8

wdma_not_m8:
    // dispatch to M=4/16/32/64 paths
```

ag_rDMA.s

```
ag_rDMA
rdma_check_w32:
// dispatch to W=32 exact path
j    rdma_w32_fast

rdma_check_w24:
// dispatch to W=24 exact path
j    rdma_w24_fast

rdma_check_w28:
// dispatch to W=28 exact path
j    rdma_w28_fast

rdma_check_w36:
// dispatch to W=36 exact path
j    rdma_w36_fast

rdma_check_w40:
// dispatch to W=40 exact path
j    rdma_w40_fast
```

-ag_wAGU, ag_wDMA에서 가장 자주 나오는 M=8을 branch not-taken 경로로 두어 분기 비용을 줄임.

-ag_rDMA는 width check 순서를 $32 \rightarrow 24 \rightarrow 28 \rightarrow 36 \rightarrow 40 \rightarrow \dots$ 식으로 재배치해 자주 나오는 width를 더 빨리 만나게 함.

Result screenshot

```
[sweep] T = (1/N) sum E_n/ALB_n = 3.6980 (N=168)
[sweep] finished: 168/168 passed, 168/168 completed
Warning: some scml2 stream(s) not destroyed before infrastructure shutdown. $
environment variable SNPS SLS VP_SCML2_LOGGING VERBOSE WARNINGS for more deta
```

$T=3.698$

Idea	Speedup (culmulative)	T
Baseline	1x	6.1545
Idea9	1.013x (1.64)	3.748
Idea10	1.011x (1.66)	3.706
Idea11 (this slide)	1.002x (1.664)	3.698

Assembly_code review

Idea12: wDMA의 row_len=224 경로를 2-row block loop로 변경

ag_rAGU.s

```
ag_rAGU
ragu_check_c32:
// dispatch to 3x3 C=32 exact path
j    ragu_3x3_c32_fast

ragu_check_c2:
// dispatch to 3x3 C=2 exact path
// guard: S==3 and R==3
j    ragu_3x3_c2_fast
```

ag_rDMA.s

```
ag_rDMA
addi x7, x10, -16
bnez x7, rdma_check_w8

// dispatch to W=16 hot path
j    rdma_w16_fast

rdma_check_w8:
// dispatch to W=8 exact path
j    rdma_w8_fast

rdma_prepare_non8_non16:
// dispatch to other width paths
```

ag_wDMA.s

```
ag_wDMA
wdma_rowlen224_fast:

// process 2 rows per iteration
// reduce loop-control overhead

wdma_row_loop:
...
// copy row 0

...
// copy row 1

addi x12, x12, 2
bltu x12, x3, wdma_row_loop
```

-ag_rAGU는 기존 C=4/8/16/32 외에 S=3, R=3, C=2 전용 path를 추가하여 generic loop를 우회
-ag_rDMA는 row_len=16 호출 비중이 높다고 보고 W=16을 branch not-taken hot path로 배치
-ag_wDMA는 wdma_rowlen224_fast를 1-row loop에서 2-row block loop로 변경하여 loop branch 및 counter 업데이트 오버헤드를 줄임

Result screenshot

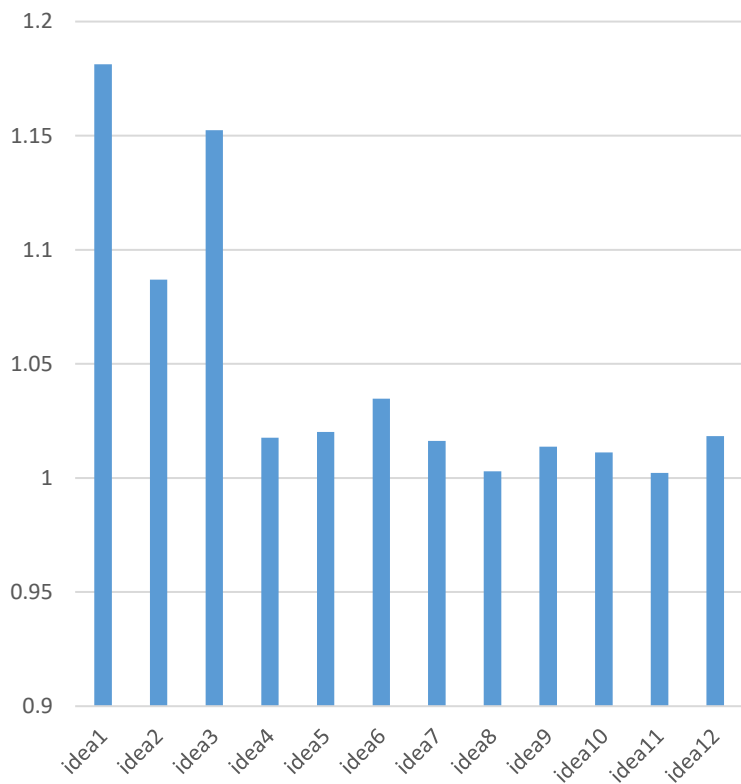
```
[sweep] T = (1/N) sum E_n/ALB_n = 3.6310 (N=168)
[sweep] finished: 168/168 passed, 168/168 completed
Warning: some scml2 stream(s) not destroyed before infrastructure
environment variable SNPS_SLS_VP_SCML2_LOGGING_VERBOSE_WARNINGS
[student14@students mp26_termproject]$ cd simulator/
[student14@students simulator]$ awk -F, 'NR>1 {sum[$1]+=$16; cnt[$1]++;}
(l in sum) printf "Layer %s avg = %.6f\n", l, sum[l]/cnt[l]}'
-n -k2
Layer 0 avg = 3.631050
Layer 1 avg = 3.599567
Layer 2 avg = 4.507563
Layer 3 avg = 2.831534
Layer 4 avg = 3.566033
Layer 5 avg = 2.555632
Layer 6 avg = 2.773231
Layer 7 avg = 2.355140
Layer 8 avg = 2.512760
[student14@students simulator]$
```

T=3.631

Idea	Speedup (culmulative)	T
Baseline	1x	6.1545
Idea10	1.011x (1.66)	3.706
Idea11	1.002x (1.664)	3.698
Idea12 (this slide)	1.018x (1.69)	3.6310

Performance of Optimization Ideas

compare performance



idea1	1.181241
idea2	1.086885
idea3	1.152332
idea4	1.017588
idea5	1.020189
idea6	1.034784
idea7	1.016271
idea8	1.002974
idea9	1.013661
idea10	1.011224
idea11	1.002271
idea12	1.018452

Best idea : idea1

기존 baseline과 generic 연산 구조를 rAGU/rDMA/wAGU/wDMA assembly를 재작성하여 주소 계산과 load/store 흐름을 명령어 단위로 제어함으로써 불필요한 control overhead 감소

Best인 이유: 모든 호출에 영향을 주는 전체적인 구조 최적화였기 때문이다.

Worst idea: idea11

fall-through로 재배치 &rDMA width dispatch 순서를 조정

Worst인 이유: 새로운걸 추가하지 않고 branch순서 쏘는 fall-through를 조정했기 때문에 상대적으로 개선이 적었다. 또한 이미 최적화가 꽤 진행된 상태가 되며 전체에서 T에 영향을 미치는 비중이 감소하였기 때문이다.